

Open-Jev Judgments on CallScreenBench: Calibrated One-Pass Scam Screening with a Small Language Model

Simiao Ren[†] Kidus Zewde* Xingyu Shen* Yuchen Zhou* Dennis Ng*
Ankit Raj* Tommy Duong* Yuxin Zhang* Neo Tiangratanakul*

Scam.ai (Reality Inc.)

*Equal contribution. [†]Corresponding author: benren@scam.ai

Abstract

Screening a phone call for fraud needs a trustworthy probability after every caller turn, in milliseconds. Jev-style typed decisions promise exactly that: declared options go in, one calibrated probability per option comes out of a single forward pass, with no generated text. We test an open implementation of this readout, JevLite, on scam-call screening: Qwen3-4B is LoRA-tuned so that the temperature-scaled softmax over two answer-label logits is $P(\text{scam})$. On 41 held-out CallScreenBench scenarios (577 per-turn decisions) a three-seed ensemble reaches AUROC .974 with calibration error .052, non-inferior to an LLM judge (MiniMax-M3) at a pre-registered .02 margin, with no false alarms on legitimate calls, decisions 1.14 turns earlier under the same hang-up rule, and 64.5 ms per decision on one consumer GPU, 4.9 $\times$ lower than the same backbone fine-tuned to generate its answer. The gain is in the readout and calibration, not accuracy: a fine-tuned ModernBERT encoder is not significantly worse, the recipe was selected with test-set exposure, and all callers are synthetic. We claim no architectural novelty; the contribution is the application and an evaluation reporting calibration, false alarms and decision timing alongside AUROC.

1 Introduction

Phone scams are increasingly screened by software that answers on the user’s behalf, from on-device assistants to research prototypes of delegated “call secretaries” [9, 31]. The callers are increasingly automated too: voice-enabled LLM agents carry out common scams end to end [8], more than a quarter of unwanted calls in a 66-day honeypot deployment opened with a recorded or synthetic voice [36], and real scam calls reveal their intent early [40]. A screener must therefore reach a judgment that software can act on—hang up, take a message, or put the call through—early in the call, within

hundreds of milliseconds, and without hanging up on the user’s bank or doctor [9].

Current scam-call work either prompts a large LLM to classify the call [37, 38] or fine-tunes a small decoder to *generate* a structured answer that is then parsed [11, 42]. Both treat the decision as text: generated scores are not calibrated probabilities unless trained to be [3, 23, 24], small models can fail to produce parseable output [11], and calibration and false alarms on legitimate calls are rarely reported (Section 2).

Jev [2] replaces generated text with typed decisions—a state and questions with declared options go in, one probability per option comes out—and open reimplementations obtain the same interface from ordinary decoders by reading answer-label logits in one forward pass [21, 49]. Is that interface a good fit for call screening, where decisions must be fast, repeated at every turn, and trustworthy as probabilities?

We make no architectural claim: JevLite follows the open reimplementations, and label-token readout itself is established [17, 35, 50]. Our contribution is the application and its evaluation (Figure 1): we (i) adapt the readout to per-turn scam screening, fine-tuning it with teacher-labeled intent questions; (ii) evaluate it on held-out CallScreenBench scenarios against an LLM judge [52], a fine-tuned ModernBERT-large encoder [44], a Qwen3 fine-tune of the same backbone and the untuned backbone, reporting calibration, false alarms on legitimate calls and decision timing alongside AUROC, with scenario-level confidence intervals and seed replication; and (iii) report where the approach fails: ambiguous calls, mid-utterance decisions, prompt sensitivity, and a recipe selection that was not blind to the test set.

(a) Qwen3 fine-tune, generated answer (prior scam-call work)



(b) JevLite: Jev-style typed decision (this work)

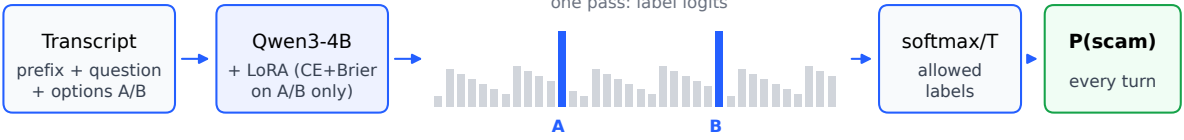


Figure 1: (a) Prior scam-call classifiers fine-tune a decoder with a language-modeling loss over a generated answer and parse the output. (b) JevLite uses the same kind of decoder but restricts training and readout to the declared answer labels: one forward pass yields a softmax over the labels, which a temperature fitted on validation data turns into a calibrated $P(\text{scam})$ after every caller turn.

2 Related Work

Scam-call detection. Datasets range from synthetic English dialogues, which keyword baselines nearly solve [37], to Chinese audio-text corpora [26, 42], streaming labels for Indian-English calls [1], and real honeypot calls [40]. Methods range from prompted judges with per-utterance warnings [38] and reinforcement-learned audio LLMs [42] to turn-by-turn tactic prediction [20] and instruction-tuned small models that re-estimate risk each turn [11]. These works report accuracy or F1; none of the LLM-based ones reports calibration, and false alarms on legitimate callers are rarely measured. LLM-paraphrased attacks on vishing classifiers [22] and automated voice-agent scams [8] show that the threat itself is adapting.

LLMs as classifiers and calibration. Scoring answer labels with next-token probabilities predates instruction tuning [6, 34, 35]; restricting the softmax to the allowed labels and correcting the residual label bias is contextual calibration [14, 50], and Llama Guard and ShieldGemma publish an answer token’s probability as their score [17, 48]. Whether a model’s stated confidence can be trusted is contested [18, 39, 46]; first-token probabilities can disagree with the generated answer [43], option order changes answers [32, 51], and instruction tuning pushes answer-token scores towards overconfidence [7]. Calibration is measured with expected calibration error [29, 30] and proper scoring rules [5, 12], restored post hoc by temperature scaling [13] or trained for directly [3, 23, 24]. Distilling a large judge into a small model [15, 19], fine-tuned encoders for classification [44, 45], early risk de-

tection on text streams [25] and selective prediction [10] are the other threads we draw on.

Jev and open reimplementations. The vendor describes Jev as a closed “System One” model with a “parallel sampler”, trained by “reinforcement learning for calibrated decisions” [2]; none of this is public, and no independent evaluation of Jev exists at the time of writing. Open attempts rebuild the interface on frozen decoders [21, 49], diffusion language models [27], fine-tuned decoders [4] and encoders [28]; reading answer letters matches grammar-constrained JSON, while model-written probabilities collapse [33]. None has been evaluated on scam calls or on calibrated per-turn decisions.

3 Method

Typed decisions. A decision is a question q with K declared options, each rendered with a single-token label $\ell_1, \dots, \ell_K$ (A, B, ...). Given a transcript prefix x , the prompt $(x, q, \text{options})$ (Appendix A) is run through the decoder once, and the logits z_k of the label tokens at the last position are normalized over the allowed labels only [14, 50], with a temperature T fitted on validation data (below):

$$p_k = \frac{\exp(z_k/T)}{\sum_{j=1}^K \exp(z_j/T)}. \quad (1)$$

No text is generated, so the output is always one of the declared options. For the main question, *is this caller attempting a scam or fraud rather than a legitimate call?*, $K = 2$ and $P(\text{scam}) = p_A$.

Three ways to read the same decoder. Every fine-tuned Qwen3 system in this paper is Qwen3-4B with LoRA adapters; what differs is the training target, the position that is read, and whether decoding happens (Table 1). (i) *Parsed label*, as in prior scam-call work: next-token loss over a JSON answer, which is generated token by token and parsed; the output can be malformed and carries no probability, since a number written into the text is more generated text. (ii) *Label-token readout*: the same weights, but the answer prefix {"scam": " is teacher-forced and the renormalized, temperature-scaled probabilities of the yes/no tokens are read in one pass; the training objective never targeted this distribution directly. (iii) *Typed decision* (JevLite, following Jev): the options are declared as single-token labels and the loss (cross-entropy plus Brier) is applied only to those label logits, normalized over the allowed labels (Eq. 1). The quantity trained is exactly the quantity read: one forward pass, an output restricted to the declared options by construction, and a probability whose calibration is part of the objective.

Training. We attach LoRA adapters [16] (rank 16, $\alpha = 32$, all attention and multi-layer perceptron (MLP) projections; Appendix A) to Qwen3-4B [47] and train on the label logits only, with the sum of cross-entropy (CE) and the Brier score over the allowed-label softmax ($T = 1$ during training). Every caller-turn prefix of a training call is a training example labeled with the call’s ground truth, so the model is trained to answer with the evidence available so far. We add two auxiliary intent questions—whether the caller pushes their own callback channel or discourages independent verification, and whether they ask for a secret such as a PIN or one-time code—whose labels come from MiniMax-M3 as a teacher on training and validation prefixes only, discarding teacher probabilities between .3 and .7. Gray scenarios whose ground truth is legitimate receive a loss weight of 2. The recipe trains for one epoch at learning rate 5×10^{-5} .

Calibration and aggregation. One temperature T per model is fitted by minimizing negative log-likelihood on validation prefixes. We train three models that differ only in seed and average their temperature-scaled $P(\text{scam})$; this ensemble is what Table 2 calls JevLite. The seeds’ logits correlate at $\geq .987$ and their AUROCs span .969–.977. A deployed screener would use a single model, and

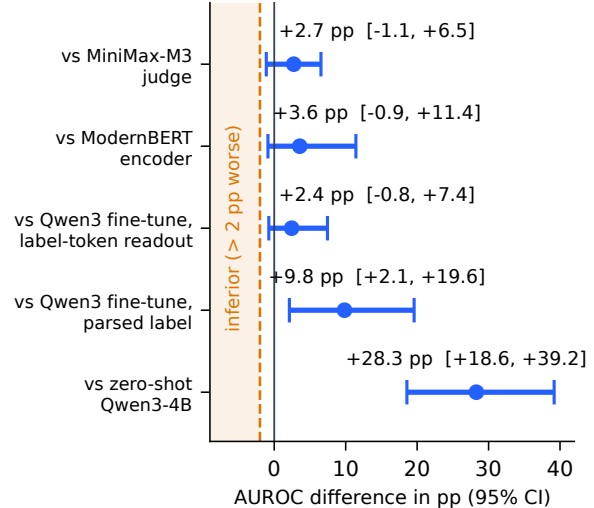


Figure 2: Paired AUROC differences in percentage points (pp), with 95% scenario-bootstrap intervals. A difference below the dashed line (2 pp worse) would fail the pre-registered non-inferiority test. The Qwen3 fine-tune is the same backbone trained with the ordinary next-token loss to write a JSON answer (arm A), read either by parsing its label or from its label-token probability.

all latencies refer to one model.

Streaming use. Screening re-asks the question after every caller turn. Because questions about the same call share the transcript prefix, the prefix is encoded once and each question adds only its short suffix to the cached state.

4 Experimental Setup

Data. CallScreenBench [9] provides 48 core scenarios (24 scam, 15 legitimate, 9 gray, where gray scenarios have a designated scam or legitimate ground truth) with transcripts of an LLM caller (MiniMax-Text-01) talking to 16 different secretary agents. We split the core scenarios by scenario into 40 for training (3,187 turn-prefix decisions) and 8 for validation (650); the validation set is small and underpins every temperature and threshold in the paper. For testing we generate new calls for the 41 evaluation scenarios that do not appear in the core set, with the benchmark’s own caller and two scripted secretaries (one that stalls, one that complies), giving 82 calls and 577 turn-end decisions. No test scenario occurs in training or validation. Test scenarios are new instances of the benchmark’s scam archetypes, not new archetypes (Appendix B).

Readout	Training loss	At inference	Forward passes	Output space	Probability
Parsed label	next-token loss over the whole answer text	generate the JSON answer, then parse it	one per generated token (~ 8)	free text; may be malformed	none (a written number is more text)
Label-token readout	same weights as above	teacher-force { "scam": " ", read the next-token distribution	one	yes/no tokens by renormalization	post hoc: renormalize, then fit T on validation
Typed decision (JevLite)	CE + Brier on the label logits only, normalized over the declared options	read the label logits at the question	one	declared options, by construction	trained directly; T fitted on validation

Table 1: Three ways to read one fine-tuned decoder. All three share the weights (Qwen3-4B with LoRA), the transcript rendering and the question; they differ only in what the loss targets, which position is read, and whether the model must decode. Prior scam-call classifiers use the first row; JevLite and the open Jev reimplementations use the third; the second is how the first row’s model can be read without changing its training.

Baselines. (1) *The LLM judge*: MiniMax-M3, the judge of our voice-honeypot pipeline [40], prompted on the same prefixes to return a probability in JSON following TypeSafe’s official LLM adapter [41]; its prompt was not tuned, and a discrete yes/no variant reaches only AUROC .823. (2) *The encoder*: ModernBERT-large [44] fine-tuned as a binary classifier on the same training prefixes, epoch and temperature chosen on validation. (3) *The encoder with auxiliary heads*: the same encoder with two extra heads on the teacher-labeled intent questions JevLite receives, so that supervision is matched. (4) *The Qwen3 fine-tune*: the same backbone trained with the ordinary next-token loss to write { "scam": "yes" }, read by parsing or from its label token (Table 1, Section 5). (5) *Zero-shot*: untuned Qwen3-4B read out exactly as JevLite.

Metrics. We report AUROC, accuracy at $P = .5$, expected calibration error [ECE; ten equal-width bins, 29] and Brier score [5, 12], plus accuracy by call family. All trained systems and the zero-shot backbone are temperature-scaled on validation; the judge’s verbalized probability is used as emitted and, in the row so marked, temperature-scaled as well. Intervals are 95% percentile intervals from 2000 bootstrap resamples of test *scenarios*, since decisions within a scenario are correlated; paired differences are resampled jointly (Appendix C), and the turn difference resamples scenarios. ECE intervals are upward-biased at this sample size, so only paired ECE differences are reported; no multiplicity correction is applied. The success criterion was written down before any training run: “beats

the judge on AUROC and ECE, or matches AUROC within 0.02 with at least $20\times$ lower latency”; the .02 margin is a convention, not derived from an operating-point cost. The superiority arm was not met, so we report the non-inferiority arm, a lower confidence bound on Δ AUROC above $-.02$; the latency clause is met against the judge’s API deployment, not against a local Qwen3 fine-tune of the same backbone ($4.9\times$). Latency is the per-decision median (p50) and 95th percentile (p95) on one RTX 3090 at batch size 1, with no other job running for the JevLite measurement; the judge’s is remote API wall time under 8 concurrent requests, including about 108 reasoning tokens per decision.

Model selection. About ten recipes had been scored on the test set before the selection rule was written: “best mean test AUROC across at least two seeds among recipes with legitimate-call accuracy at least .97”. Only two recipes (the intent-question recipe (i7) and the two-epoch LoRA recipe (lora_r16)) were eligible; the statistic is the mean single-seed test AUROC (.972 against .930), not the ensemble AUROC of Table 3. The headline recipe is therefore chosen with test-set exposure and its point estimate is subject to winner’s-curse inflation. On the 8-scenario validation set, accuracy favored seed 0 of the chosen recipe (.888 against .865); Table 3 lists validation AUROC and accuracy for every recipe scored.

5 Results

Ranking and calibration. Table 2 and Figure 2 summarize the comparison. Fine-tuning the readout is what makes it work: JevLite improves over

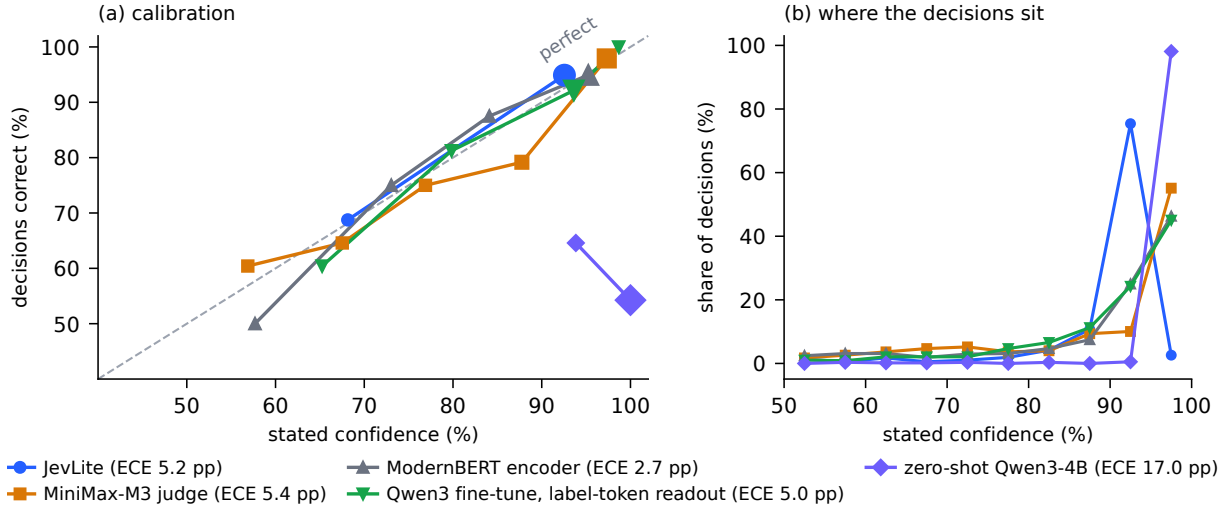


Figure 3: (a) Calibration on the test set, in percentages, including the Qwen3 fine-tune of the same backbone read from its label-token probability (arm A). Bins hold equal numbers of decisions and are merged until no two points lie within 5 percentage points of each other; marker area grows with the decisions behind each point, and points below the diagonal are over-confident. (b) How the decisions are distributed over stated confidence: all trained systems concentrate above 90%, which is why bin choice matters [30]. ECE in Table 2 uses the conventional equal-width bins.

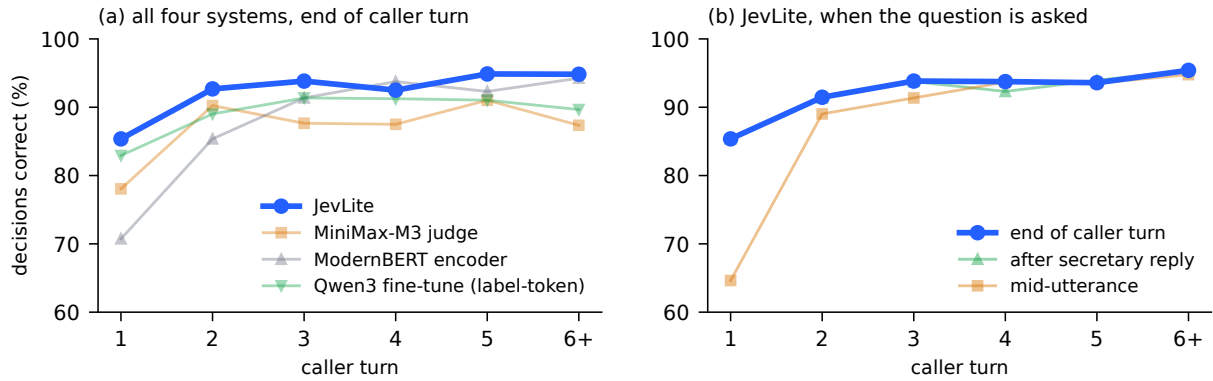


Figure 4: Accuracy by caller turn. (a) JevLite, the LLM judge, the encoder and the Qwen3 fine-tune of the same backbone (arm A, label-token readout) on the same turn-end decisions. (b) The cost of asking earlier within a turn, for seed 0 of the JevLite recipe scored on streaming prefixes.

the same backbone read out zero-shot by $+0.283$ AUROC $[.186, .392]$. Against the LLM judge the ensemble’s difference is $+0.027$ $[-.011, .065]$; the lower bound clears the $-.02$ margin by $.009$, so the ensemble is non-inferior but not better. The single seeds are closer to the line $(+0.031$ $[-.005, .067]$, $+0.023$ $[-.018, .063]$, $+0.024$ $[-.018, .064]$; all three clear the margin, two of them by a few thousandths, so a deployed single model sits at the edge of the criterion. Non-inferiority is moreover established against one judge, MiniMax-M3, which belongs to the model family that wrote the calls and supplied the auxiliary labels. Against the Qwen3 fine-tune (Section 5) the difference is $+0.098$ $[.021, .196]$ when that model is read by parsing its label

and only $+0.024$ $[-.008, .074]$ when it is read the Jev way from its label-token probability. JevLite is not significantly better than the encoder $(+0.036$ $[-.009, .114])$, which has the lowest ECE, nor than the encoder with matched auxiliary heads $(+0.026$ $[-.023, .105])$, the fair comparison since it sees the same teacher-labeled questions. Calibration is comparable to the judge (ECE difference $-.002$ $[-.048, .036]$; Figure 3) and remains so after temperature-scaling the judge on validation (ECE $.049$, Brier $.096$; paired ECE difference $+0.003$ $[-.056, .030]$). All trained systems lie close to the diagonal, the encoder most closely (ECE $.027$), whereas the untuned backbone is badly over-confident (ECE $.170$): answer probabilities read from a model not trained

	JevLite (3-seed ens.)	LLM judge (MiniMax-M3)	ModernBERT	ModernBERT + aux heads	Qwen3 FT parsed label	Qwen3 FT label-token	Zero-shot
AUROC (%)	97.4	94.7	93.8	94.8	87.5	95.0	69.1
Accuracy (%)	92.7	87.0	88.9	<u>90.6</u>	89.2	89.2	55.1
ECE (%) ↓	5.2	5.4	2.7	<u>4.2</u>	10.7	5.0	17.0
ECE, judge T -scaled (%)	—	4.9	—	—	—	—	—
Brier ($\times 100$) ↓	6.1	9.5	8.2	<u>6.9</u>	10.7	8.1	28.0
JevLite, mean of 3 single seeds: AUROC 97.2%, accuracy 92.3%, ECE 4.7%, Brier 6.3							
Legit ($n=12$, %)	100.0	82.3	100.0	100.0	100.0	100.0	<u>90.2</u>
Scam ($n=21$, %)	99.3	<u>95.5</u>	92.4	92.1	99.3	99.3	42.6
Gray/scam ($n=5$, %)	82.5	<u>85.0</u>	85.0	91.2	72.5	72.5	21.2
Gray/legit ($n=3$, %)	38.1	<u>50.0</u>	28.6	42.9	9.5	9.5	69.1
p50 latency (ms) ↓	64.5	1,946 [†]	<u>29.7</u>	14.0 [‡]	317	— [§]	64.5 [*]

Table 2: Held-out results on 577 turn-end decisions from 41 unseen scenarios (n : scenarios per family), in %; Brier is $\times 100$. JevLite is a three-seed ensemble (AUROC 95% CI [93.3, 99.7] %); all other systems are single models. Best value per row bold, runner-up underlined (ties share the mark); lower is better where marked ↓. Paired differences against JevLite are given in the text and Figure 2. “ECE, judge T -scaled” temperature-scales the judge’s probability on validation ($T = 1.49$), as the other systems already are. The two Qwen3 fine-tune (FT) columns are one generatively trained model (arm A, base recipe, single seed) read two ways (Table 1). [†]Remote API wall time under 8 concurrent requests, about 108 reasoning tokens per decision. ^{*}Same forward pass as JevLite. [§]One forward pass, not timed separately. [‡]Measured in a separate session; the single-task encoder’s 29.7 ms was measured under different host load, so the two encoder numbers differ although the architectures are identical. The encoder with auxiliary heads used its best validation epoch (2) and $T = 2.9$.

for the readout are confident but unreliable.

False alarms on legitimate calls. The clearest difference is on legitimate callers. JevLite and both encoders classify all legitimate decisions correctly, while the judge classifies .823 of them correctly (difference +.177 [.008, .360]); JevLite is also more accurate than the judge on plain scams. This is the only paired difference against the judge whose interval excludes zero, it rests on 12 scenarios, and we treat it as exploratory. The judge is better on gray calls, but with three gray/legit and five gray/scam test scenarios these differences are not significant.

Owner-facing decisions. To show what a screener would do, we replay a candidate rule that hangs up once two consecutive caller turns reach $P(\text{scam}) \geq \tau$, with τ fitted on validation separately per system (0.92 for JevLite, 0.97 for the judge). Both systems hang up on a similar share of scam-side calls (40/52 against 41/52) and on no legitimate call; JevLite reaches the decision earlier (Figure 5), at the cost of one fewer detection: on the 36 calls where both hang up it decides 1.14 turns earlier on average [0.55, 1.67], earlier on 25, at the same turn on 6 and later on 5. The rule is an illustration: we do not choose an operating point, and the counts move with small changes of

T (Appendix D).

Latency. One decision takes 64.5 ms at the median (p95 110.5 ms): roughly $30\times$ faster than the judge through its API at the median and $52\times$ at p95 (Figure 6), a ratio that mixes model size with network queueing, chain-of-thought generation and a longer prompt. The like-for-like local comparison is the Qwen3 fine-tune of the same backbone, which must decode a JSON answer: 317 ms at the median, about $4.9\times$ slower than the readout. Each additional question on the same call adds 21.8 ms on the shared prefix; five questions take 168.7 ms instead of 321.6 ms when each is encoded afresh, with the argmax changing in 6/1500 decisions. The encoder is faster still (29.7 ms) but answers only the questions it was trained for.

When in the call. Accuracy at the end of the first caller turn is already .854 and reaches .927 by the second (Figure 4). Asking right after the secretary speaks costs nothing, but asking in the middle of a caller’s utterance does: for seed 0 alone scored on streaming prefixes (the ensemble was not scored this way), mid-utterance accuracy is .888 against .927 at turn end, a drop of 3.8 pp [2.2, 5.6] concentrated on the first turn.

Recipe and robustness. Table 3 shows that the shorter, lower-learning-rate recipe drives the AU-

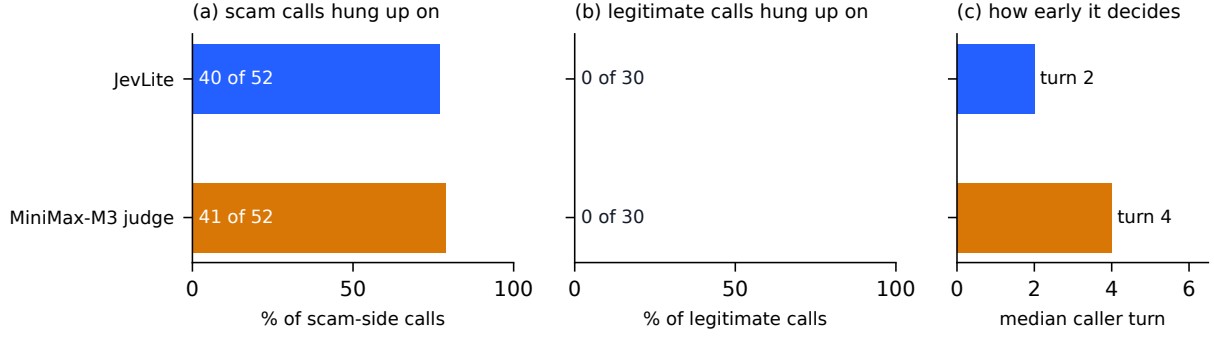


Figure 5: Replay of the candidate rule “hang up once two consecutive caller turns read $P(\text{scam}) \geq \tau$ ” on the 41 held-out scenarios ($\tau = 0.92$ for JevLite, 0.97 for the LLM judge). Denominators are calls (two per scenario): 52 scam-side calls (scam and gray/scam) and 30 legitimate-side calls (legitimate and gray/legit). (a) Scam-side calls the screener ends; higher is better. (b) Legitimate-side calls it wrongly ends; zero is the goal. (c) How far into the call the decision is made; earlier is better.

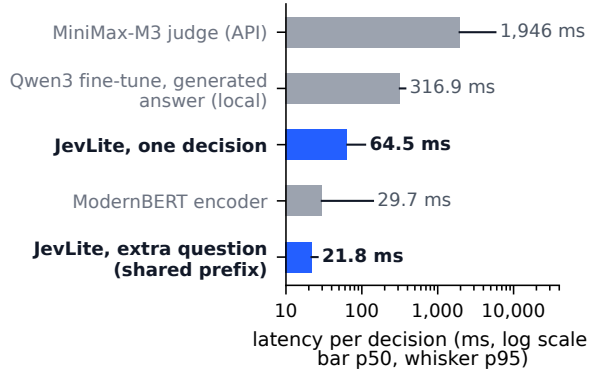


Figure 6: Latency per decision (log scale; bar p50, whisker p95) on one RTX 3090 for JevLite, the encoder and the Qwen3 fine-tune of the same backbone (which must decode a JSON answer), versus the LLM judge measured through its API under concurrent requests. “Extra question” is the marginal cost of one more question on the shared prefix.

ROC gain, while the gray weight and the intent questions nominally add accuracy within seed noise. Robustness is imperfect: averaging seeds 0 and 1, swapping the order of the answer options flips 2.6% of decisions (95% interval [0.2, 5.8]%), in line with known option-order sensitivity [32, 51], and a held-out paraphrase of the question flips 5.2% ([2.0, 9.4]%), 28 of 30 towards “legitimate”. On questions never seen in training, the fine-tuned model improves over zero-shot on extraction-like questions (a stated amount, +.013 [.002, .033], both models near ceiling; a callback number, +.041 [.020, .075]) and not on judgment-like ones (urgency −.062 [−.133, .013], sympathy −.113 [−.230, .001]; Appendix D).

Recipe	Test					Validation	
	AUROC	Acc	ECE ↓	Gray/scam	Gray/legit	AUROC	Acc
2 ep, $1e-4$ ‡	93.4	91.2	5.1	70.0	35.7	93.9	88.6
1 ep, $5e-5$	97.2	89.4	5.0	78.7	9.5	100.0	85.9
+ gray weight	97.2	89.9	9.8	77.5	9.5	99.8	85.9
+ intent Qs (JevLite)‡	97.4	92.7	5.2	82.5	38.1	99.9	87.2

Table 3: Recipe ablation, all values in %; the best value in each column is bold. **Takeaway: the shorter 1-epoch, lower-learning-rate recipe delivers almost all of the AUROC gain (+3.8 pp); the gray weight and the teacher-labeled intent questions then add accuracy, not ranking.** ‡Three-seed probability ensembles (single-seed test AUROC means 93.0% and 97.2%, the statistic the selection rule used); the other rows are single runs. These recipes were scored on test before the selection rule was written (Section 4). Gray-family accuracies rest on 5 and 3 scenarios and vary across seeds by up to about 30 pp, so gray-call differences are within seed noise.

Interface versus fine-tuning. To separate the fine-tuning from the readout we train matched arms on the same backbone, data, LoRA budget and schedule at the base recipe (Table 3, row 2: seed 0, no gray weight, no intent questions), which is not the headline recipe: (A) the Qwen3 fine-tune of the ordinary kind, read by parsing its label, by a verbalized confidence the prompt asks for (a prompt unseen in training), and from the label-token probability at the teacher-forced JSON prefix, as Shield-Gemma does [48]; (B) the typed-decision readout, with and without the Brier term; and (C) a linear classification head on the same backbone. Table 4 in Appendix D reports the arms.

Three findings follow. First, the fine-tuning, not the interface, buys the accuracy: the parsed label

is as accurate as the base-recipe readout (.892 vs. .894) and the classification head is at least as good (.927, AUROC .962). Second, the ordinary fine-tune does not yield a trustworthy probability by itself: its parsed label scored as 0/1 has ECE .107, and when asked for a confidence it writes 0.95 or 1.0 on almost every decision, ranking calls barely better than the label does (AUROC .914). Third, reading the *same* fine-tune the Jev way, from the label token with a validation-fitted temperature, recovers a well-ranked and calibrated score (AUROC .950, ECE .050), only slightly below the model trained for that readout (.972); what it still lacks is the format guarantee and the latency (317 vs. 63.6 ms per decision). Across two seeds the Brier term buys a little calibration (ECE .050 / .021 vs. .076 / .051 for CE+Brier vs. CE only, seeds 0 / 1) and no ranking (AUROC .972 / .967 vs. .970 / .966). The readout is therefore a training and readout choice on an ordinary decoder whose value is a guaranteed output space, a usable probability and a single forward pass, not higher accuracy.

6 Discussion

For a fixed question such as *is this a scam?*, a small decoder with the readout can stand in for an LLM judge here: ranking is on par, legitimate callers are spared, decisions come earlier, and each costs tens of milliseconds on one consumer GPU rather than seconds through an API. But the fair comparison is the encoder with matched auxiliary heads, which has nominally lower ECE (.042 vs. .052) but higher Brier (.069 vs. .061), is faster, and is not significantly worse in ranking (JevLite – encoder +.026 [−.023, .105]; a wide, underpowered interval, so absence of evidence rather than equivalence). What remains specific to the decoder readout is that questions can be posed in natural language at run time, for about 21.8 ms each on the shared prefix, and our unseen-question results show that this buys something only on extraction-like questions. Our recommendation is therefore plain: for a fixed question, use the encoder; the decoder readout is worth its cost only when questions must be posed at run time, and we did not show that any run-time question improves a screening outcome. Judgment-like questions and ambiguous gray calls remain the weak points, where an LLM judge, or a selective-prediction cascade that defers uncertain calls to one [10], is still preferable.

7 Conclusion

We applied the typed-decision readout to scam-call screening and evaluated it with calibration, false alarms, and timing in view. A LoRA-tuned 4B model read out in one forward pass matches an LLM judge on held-out CallScreenBench scenarios at a fraction of the latency, without the judge’s false alarms on legitimate calls, but the recipe was selected with test exposure, single seeds clear the non-inferiority margin only narrowly, and an encoder with matched auxiliary labels is not significantly worse. Future work should test real callers with independent labels, for which our honeypot corpus of over ten thousand calls [40] is the natural next step, hold out entire scam archetypes, and study whether calibration fitted on complete calls transfers to partial ones.

References

- [1] Rishi Ahuja, Kumar Prateek, and Simranjit Singh. 2026. [ICFD-31k: A large-scale dataset and benchmark for real-time conversational fraud detection](#). In *Proceedings of the Thirty-Fifth International Joint Conference on Artificial Intelligence, IJCAI-26*, pages 7065–7073. AI and Social Good track.
- [2] Diogo Almeida. 2026. [Introducing system one models & Jev](#). TypeSafe AI Blog. September 15, 2026. Accessed 2026-09-19.
- [3] David Bani-Harouni, Chantal Pellegrini, Paul Stangel, Ege Özsoy, Kamilia Zaripova, Nassir Navab, and Matthias Keicher. 2026. [Rewarding doubt: A reinforcement learning approach to calibrated confidence expression of large language models](#). In *The Fourteenth International Conference on Learning Representations (ICLR)*.
- [4] Bespoke Labs. 2026. [Bespoke Nimble: Data, model, recipe for an open Jev](#). GitHub repository. Accessed 2026-09-19, commit 35fe1f4.
- [5] Glenn W. Brier. 1950. [Verification of forecasts expressed in terms of probability](#). *Monthly Weather Review*, 78(1):1–3.
- [6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020. [Language models are few-shot learners](#). In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pages 1877–1901. Curran Associates, Inc.
- [7] André F. Cruz, Moritz Hardt, and Celestine Mendler-Dünnér. 2024. [Evaluating language models as risk](#)

- scores. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*.
- [8] Richard Fang, Dylan Bowman, and Daniel Kang. 2024. [Voice-enabled AI agents can perform common scams](#). *arXiv preprint arXiv:2410.15650*.
 - [9] Jiaqi Gan, Haoyuan Tang, Jamey Z. Liang, Siying Chen, Ankit Raj, Kidus Zewde, Yuchen Zhou, Yuxin Zhang, and Simiao Ren. 2026. [CallScreenBench: Benchmarking small language models as phone secretaries](#). *arXiv preprint arXiv:2608.01033*.
 - [10] Yonatan Geifman and Ran El-Yaniv. 2017. [Selective classification for deep neural networks](#). In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*. Curran Associates, Inc.
 - [11] Parviz Ghafariasl, Weimin Fu, Xiaolong Guo, and Shing I. Chang. 2026. [Incremental risk assessment of progressive elder financial scams via instruction-tuned small language models](#). *arXiv preprint arXiv:2609.00005*.
 - [12] Tilman Gneiting and Adrian E. Raftery. 2007. [Strictly proper scoring rules, prediction, and estimation](#). *Journal of the American Statistical Association*, 102(477):359–378.
 - [13] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. [On calibration of modern neural networks](#). In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR.
 - [14] Ari Holtzman, Peter West, Vered Shwartz, Yejin Choi, and Luke Zettlemoyer. 2021. [Surface form competition: Why the highest probability answer isn't always right](#). In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 7038–7051, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.
 - [15] Cheng-Yu Hsieh, Chun-Liang Li, Chih-kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alex Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. 2023. [Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes](#). In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 8003–8017, Toronto, Canada. Association for Computational Linguistics.
 - [16] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. [LoRA: Low-rank adaptation of large language models](#). In *The Tenth International Conference on Learning Representations (ICLR)*.
 - [17] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashmi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabza. 2023. [Llama guard: LLM-based input-output safeguard for human-AI conversations](#). *arXiv preprint arXiv:2312.06674*.
 - [18] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislaw Fort, and 17 others. 2022. [Language models \(mostly\) know what they know](#). *arXiv preprint arXiv:2207.05221*.
 - [19] Seungone Kim, Jamin Shin, Yejin Cho, Joel Jang, Shayne Longpre, Hwaran Lee, Sangdoo Yun, Seongjin Shin, Sungdong Kim, James Thorne, and Minjoon Seo. 2024. [Prometheus: Inducing fine-grained evaluation capability in language models](#). In *The Twelfth International Conference on Learning Representations (ICLR)*.
 - [20] Jeongmin Lee, Dongmyung Sul, Seung Yun, and Jinxia Huang. 2026. [Vishing-Tactics-Bench: Forecasting exploitation trajectories in voice phishing calls](#). In *Findings of the Association for Computational Linguistics: EMNLP 2026*. To appear.
 - [21] Theodore Lee. 2026. [SemIf \(formerly OpenJev\): Semantic ifs from open models, on a 3090 at home](#). GitHub repository. Accessed 2026-09-19, commit ca3ba65.
 - [22] Wenhao Li, Selvakumar Manickam, Yung-Wey Chong, and Shankar Karuppayah. 2025. [Talking like a phisher: LLM-based attacks on voice phishing classifiers](#). In *Digital Forensics and Cyber Crime (EAI ICDF2C 2025)*, Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, pages 503–521. Springer Nature Switzerland.
 - [23] Yibo Li, Miao Xiong, Jiaying Wu, and Bryan Hooi. 2025. [ConfTuner: Training large language models to express their confidence verbally](#). In *Advances in Neural Information Processing Systems (NeurIPS)*.
 - [24] Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. [Teaching models to express their uncertainty in words](#). *Transactions on Machine Learning Research*.
 - [25] David E. Losada and Fabio Crestani. 2016. [A test collection for research on depression and language use](#). In *Experimental IR Meets Multilinguality, Multimodality, and Interaction: 7th International Conference of the CLEF Association, CLEF 2016*, volume 9822 of *Lecture Notes in Computer Science*, pages 28–39. Springer.
 - [26] Zhiming Ma, Peidong Wang, Minhua Huang, Jinpeng Wang, Kai Wu, Xiangzhao Lv, Yachun Pang, Yin Yang, Wenjie Tang, and Yuchen Kang. 2025. [TeleAntiFraud-28k: An audio-text slow-thinking dataset for telecom fraud detection](#). In *Proceedings of the 33rd ACM International Conference on Multimedia (MM '25)*, pages 5853–5862, Dublin, Ireland. Association for Computing Machinery.

- [27] Matt Mastracci. 2026. [\[Core\] structured generation mode for DiffusionGemma model \(Jev-like\)](#). vLLM pull request #57250. Unmerged pull request, opened 2026-09-16. Accessed 2026-09-19.
- [28] Nandakishor Mukkunoth. 2026. [Laya: Multilingual, non-autoregressive system 1 decision engine](#). GitHub repository. Accessed 2026-09-19, commit d113dca.
- [29] Mahdi Pakdaman Naeni, Gregory F. Cooper, and Milos Hauskrecht. 2015. [Obtaining well calibrated probabilities using Bayesian binning](#). In *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*. AAAI Press.
- [30] Jeremy Nixon, Michael W. Dusenberry, Linchuan Zhang, Ghassen Jerfel, and Dustin Tran. 2019. [Measuring calibration in deep learning](#). In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 38–41.
- [31] Sharbani Pandit, Krishanu Sarker, Roberto Perdisci, Mustaque Ahamad, and Diyi Yang. 2023. [Combating robocalls with phone virtual assistant mediated interaction](#). In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 463–479, Anaheim, CA. USENIX Association.
- [32] Pouya Pezeshkpour and Estevam Hruschka. 2024. [Large language models sensitivity to the order of options in multiple-choice questions](#). In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 2006–2017, Mexico City, Mexico. Association for Computational Linguistics.
- [33] Mikhail Rakutko. 2026. [mini-Jev: Read the letter](#). GitHub repository. Accessed 2026-09-19, commit ca61219.
- [34] Joshua Robinson, Christopher Michael Rytting, and David Wingate. 2023. [Leveraging large language models for multiple choice question answering](#). In *The Eleventh International Conference on Learning Representations (ICLR)*.
- [35] Timo Schick and Hinrich Schütze. 2021. [Exploiting cloze-questions for few-shot text classification and natural language inference](#). In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 255–269, Online. Association for Computational Linguistics.
- [36] Xingyu Shen, Tommy Duong, Muduo Xu, Xiaodong An, Jiaqi Gan, Haoyuan Tang, Jamey Z. Liang, Siyu Zhang, Yan Zhang, Ethan Traister, and Simiao Ren. 2026. [The machines are calling: Measuring automated and synthetic voices in unwanted inbound calls](#). *arXiv preprint arXiv:2609.11137*.
- [37] Zitong Shen, Kangzhong Wang, Youqian Zhang, Grace Ngai, and Eugene Y. Fu. 2024. [Combating phone scams with LLM-based detection: Where do we stand?](#) *arXiv preprint arXiv:2409.11643*.
- [38] Zitong Shen, Sineng Yan, Youqian Zhang, Xipu Luo, Grace Ngai, and Eugene Yujun Fu. 2025. [“it warned me just at the right moment”: Exploring LLM-based real-time detection of phone scams](#). *arXiv preprint arXiv:2502.03964*.
- [39] Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma, Rafael Rafailov, Huaxiu Yao, Chelsea Finn, and Christopher Manning. 2023. [Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5433–5442, Singapore. Association for Computational Linguistics.
- [40] Ethan Traister, Ankit Raj, Jiaqi Gan, Xingyu Shen, Tyler Wu, Yuchen Zhou, Tommy Duong, Kidus Zewde, Siying Chen, and Simiao Ren. 2026. [Anatomy of a scam call: What 10,000 real scam and spam calls reveal about how phone scammers operate](#). *arXiv preprint arXiv:2608.24127*.
- [41] TypeSafe AI. 2026. [system-one-adapter-python: Drop-in TypeSafeClient replacement backed by LLM APIs](#). GitHub repository. Official TypeSafe adapter for the System One / Jev interface. Accessed 2026-09-19, commit adffc2e.
- [42] Peidong Wang, Zhiming Ma, Xin Dai, YongKang Liu, Shi Feng, Xiaocui Yang, Wenxing Hu, Zhihao Wang, Mingjun Pan, Li Yuan, and Daling Wang. 2026. [SAFE-QAQ: End-to-end slow-thinking audio-text fraud detection via reinforcement learning](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 26138–26157, San Diego, California, United States. Association for Computational Linguistics.
- [43] Xinpeng Wang, Bolei Ma, Chengzhi Hu, Leon Weber-Genzel, Paul Röttger, Frauke Kreuter, Dirk Hovy, and Barbara Plank. 2024. [“my answer is C”: First-token probabilities do not match text answers in instruction-tuned language models](#). In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 7407–7416, Bangkok, Thailand. Association for Computational Linguistics.
- [44] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Griffin Thomas Adams, Jeremy Howard, and Iacopo Poli. 2025. [Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2526–2547, Vienna, Austria. Association for Computational Linguistics.
- [45] Orion Weller, Kathryn Ricci, Marc Marone, Antoine Chaffin, Dawn Lawrie, and Benjamin Van Durme. 2026. [Seq vs seq: An open suite of](#)

paired encoders and decoders. In *The Fourteenth International Conference on Learning Representations (ICLR)*.

- [46] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. 2024. [Can LLMs express their uncertainty? an empirical evaluation of confidence elicitation in LLMs](#). In *The Twelfth International Conference on Learning Representations (ICLR)*.
- [47] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. [Qwen3 technical report](#). *arXiv preprint arXiv:2505.09388*.
- [48] Wenjun Zeng, Yuchi Liu, Ryan Mullins, Ludovic Peran, Joe Fernandez, Hamza Harkous, Karthik Narasimhan, Drew Proud, Piyush Kumar, Bhaktipriya Radharapu, Olivia Sturman, and Oscar Wahltinez. 2024. [ShieldGemma: Generative AI content moderation based on Gemma](#). *arXiv preprint arXiv:2407.21772*.
- [49] Eric Zhang. 2026. [openjev-sglang: Jev-compatible API endpoint based on open models \(prefill-only\)](#). GitHub repository. Accessed 2026-09-19, commit 604664a.
- [50] Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. 2021. [Calibrate before use: Improving few-shot performance of language models](#). In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 12697–12706. PMLR.
- [51] Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. 2024. [Large language models are not robust multiple choice selectors](#). In *The Twelfth International Conference on Learning Representations (ICLR)*.
- [52] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. [Judging LLM-as-a-judge with MT-Bench and Chatbot Arena](#). In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, *Datasets and Benchmarks Track*, pages 46595–46623. Curran Associates, Inc.

A Prompts and Training Details

JevLite, the ablation arms and the encoders see the same transcript rendering: caller and secretary turns, one per line, prefixed `Caller:` and `Secretary:` (the LLM judge instead receives a JSON-serialized transcript through the vendor

adapter). The system message is *You are a fast, calibrated call-screening judge*, used for the zero-shot baseline as well. The readout prompt is:

```
Phone call transcript so far
(the Secretary answers calls on
the owner's behalf):
{transcript}
Question: Is this caller
attempting a scam or fraud
against the person being called,
rather than making a legitimate
call?
A: yes
B: no
Answer with the single letter
only.
```

The Qwen3 fine-tune arm (A) replaces the last three lines with *Answer with JSON only: {"scam": "yes"|"no"} and, in its confidence variant (used at evaluation only), {"scam": "yes"|"no", "confidence": 0.0–1.0}*. The classification head (C) uses the readout prompt verbatim and ignores the answer tokens. The auxiliary intent questions used during training are *Does the caller push their own callback number or channel, or discourage verifying through an official channel?* and *Does the caller ask for a secret such as a PIN, one-time code, password or card number?*, each with the same A/B options. Transcripts longer than 6,000 characters are truncated from the left so that the most recent turns and the question are always present.

Training details. JevLite and the ablation arms attach LoRA adapters of rank 16, $\alpha = 32$ and dropout .05 to all attention and MLP projections of Qwen3-4B. Training uses an effective batch of 16 (4 per device with 4 steps of gradient accumulation), 20 warm-up steps followed by linear decay, bf16 weights and activations with the label logits read in fp32, a maximum length of 2048 tokens, and one epoch (two for the first row of Table 3). The encoders are trained at learning rate 2×10^{-5} with an effective batch of 16 for 4 epochs, the epoch chosen by validation AUROC, inputs left-truncated to 2048 tokens, and T fitted on validation. Software: Python 3.12.3, torch 2.10.0+cu128, transformers 4.57.6, peft 0.19.1. Test-call generation seed: not recorded.

B Data and Splits

CallScreenBench scenarios carry a family (scam, legitimate, gray), an archetype (22 in total), a hidden goal and a hang-up policy. Gray scenarios are written to be ambiguous and carry a designated

ground truth on one side. The 48 core scenarios are split by scenario into 40 training and 8 validation scenarios, stratified by family (4 scam, 2 legitimate, 2 gray in validation). Every caller-turn prefix of every recorded call is one decision, labeled with the scenario’s ground truth: 3,187 training and 650 validation decisions across 16 secretary agents. Test calls were generated for the 41 evaluation-only scenarios with the benchmark’s caller (MiniMax-Text-01, turn cap 8) against two scripted secretaries, giving 82 calls and 577 turn-end decisions; the streaming evaluation adds a mid-utterance prefix (the caller’s turn cut at a random word, labeled with the scenario-level ground truth like every other prefix) and an after-secretary prefix for each turn. No test scenario identifier appears in any training or validation file, and all 18 test archetypes also occur in training.

C Verification Protocol

Every number in the paper is generated from saved per-decision predictions by scripts that read those files directly; nothing is transcribed by hand. An independent checking script, separate from the training and evaluation code and to be released with it, recomputed AUROC, accuracy, Brier score and ECE from the predictions, confirmed that temperatures and policy thresholds were fitted on validation predictions only, and checked the training files of every run for test-scenario identifiers. Confidence intervals resample the 41 test scenarios with replacement (2000 draws, seed fixed); paired differences resample the same scenarios for both systems. The three training seeds of the headline recipe have test-logit correlations of at least .987 and individual AUROCs of .969–.977; a deployed model would be a single run, and all latency figures are for one.

D Additional Results

Unseen questions. Five questions never used in training were scored on the test decisions with labels derived mechanically from the scenario cards or the transcript text: whether the caller has named a specific amount of money (AUROC JevLite .996 vs. zero-shot .984), given a callback number (.999 vs. .957), whether the scenario uses urgency (.863 vs. .925) or sympathy (.447 vs. .560) as a pressure lever, and which kind of organization the caller claims to represent (six-way accuracy, unchanged). The sympathy labels come from scenario design notes and are near chance even for the zero-shot

model.

Policy sensitivity. Under the two-consecutive-turns rule the validation-optimal τ is a narrow range, and the test counts move when the fitted temperature changes by a few hundredths. We therefore report the rule as an illustration and recommend re-fitting both T and τ on deployment data before choosing an operating point.

Ablation arms. Table 4 reports the matched arms of Section 5; Table 5 gives the second seed of each arm. Every arm’s second seed lands within about one AUROC point of its first, so the ordering in Table 4 is stable across seeds; gray/legit accuracy varies more (the classification head scores .429 / .548 and the base-recipe readout .095 / .405 on gray/legit across their two seeds), as it does for every recipe in this paper.

E Ethics Statement

The study uses synthetic calls from a public benchmark and does not involve human participants or real call recordings. A screener that hangs up on legitimate callers can harm users; we therefore report false alarms on legitimate calls explicitly and do not recommend deployment without evaluation on real, independently labeled calls. The method is dual-use: a fast, calibrated $P(\text{scam})$ is also a fast oracle against which a scammer could tune a script, and our own paraphrase test moved 28 of 30 flipped decisions towards “legitimate”, the exploitable direction, in line with LLM-paraphrase attacks on vishing classifiers [22]. The code, LoRA adapters, per-decision predictions and the 82 generated test calls will be released so that such weaknesses can be studied openly; release also lowers an attacker’s cost of probing the screener, which we accept because the weakness is already exploitable by paraphrase without access to the model.

Arm	Readout	AUROC	Acc	ECE	Legit	Gray/legit	T	ms
A Qwen3 fine-tune	parsed label	.875	.892	.107	1.000	.095	–	317
A Qwen3 fine-tune	verbalized conf.	.914	.896	.069	1.000	.095	–	–
A Qwen3 fine-tune	label-token prob.	.950	.892	.050	1.000	.095	4.06	–
B readout, CE only	label logits	.970	.910	.076	.994	.095	5.24	$\approx$ 63.6
B readout (base recipe)	label logits	.972	.894	.050	1.000	.095	3.88	63.6
C head	linear head	.962	.927	.055	1.000	.429	5.18	80

Table 4: Matched ablation on the test set. All arms use the base recipe (one epoch, learning rate 5×10^{-5} , seed 0, no gray/legit weight, no intent questions), which differs from the headline JevLite of Table 2 (gray weight, intent questions, three-seed ensemble); “B readout (base recipe)” is the same run as row 2 of Table 3. Legit and Gray/legit are accuracies on the legitimate and gray/legit families. T is the temperature fitted on validation, on the same uncapped grid as the headline models; an earlier draft’s grid was capped at $T = 5$, which the CE-only and head arms hit, and reported ECEs of .065 and .042 for them. The verbalized-confidence prompt was unseen in training: arm A was trained only on the yes/no target. Arm A produced 0 unparseable answers out of 577 and omitted a confidence value on 0 when asked for one. Arms B and C read the same forward pass; arm A must decode, and its three readouts share one decode (“–” in the ms column). Latency is the median per decision on one RTX 3090. Arm-B latency was measured on the arm-B checkpoint (p50, batch 1; each extra question adds 20.8 ms); the CE-only arm shares the architecture and was not measured separately. Second seeds of arms A, B, C and the CE-only variant are reported in Appendix D; the arms are single runs unless stated.

Arm (readout)	AUROC	Acc	ECE
	seed 0 / seed 1		
A gen. (parsed label)	.875 / .878	.892 / .894	.107 / .106
A gen. (label-token p.)	.950 / .955	.892 / .892	.050 / .036
B readout, CE only	.970 / .966	.910 / .905	.076 / .051
B readout (base recipe)	.972 / .967	.894 / .931	.050 / .021
C head	.962 / .971	.927 / .922	.055 / .037

Table 5: Two seeds of the matched ablation arms of Table 4 (base recipe); each cell reads seed 0 / seed 1.